Institut Notre-Dame Beauraing

UAA11

Le langage C

Livret d'exercices



Technique de transition

–

Option Informatique

–

Titulaire du cours : L. Embrechts

1. Exercices pratiques, premières instructions

Déclarer et typer les variables

Choisir le bon type, c'est déjà résoudre la moitié du problème. Un `int` pour ce qui se compte, un `float` pour ce qui se mesure, un `char` pour un caractère unique, un `bool` pour une réponse par oui ou par non.

1.1. Déclare, pour chaque type primitif[1], une variable contenant une valeur de ton choix. Ensuite, imprime à l'écran la valeur de chacune des variables.

1.2. Rédige les déclarations complètes en C pour les variables suivantes. Si une valeur est précisée, initialise la variable en même temps que la déclaration.

La variable contient...	Valeur initiale
Le nombre de joueurs inscrits	Pas de valeur
Le temps du meilleur tour en secondes	12.48
La lettre de l'équipe	A
Si le match a été gagné ou non	non
La taille d'un joueur en mètres	Pas de valeur
Le nombre de spectateurs cumulés sur la saison	Pas de valeur
Le numéro de maillot d'un joueur	10
La distance parcourue par un coureur en km	Pas de valeur
La température le jour du match	Pas de valeur
Le montant total des cotisations encaissées	Pas de valeur

1.3. Corrige chaque déclaration lorsque c'est nécessaire :

```c
int a = 3.14;
float b = 42;
char c = "A";
double d = 5.5;
char e = 'Z';
float f = 3.14;
double g = 10;
char h = 123;
int i = 2147483648;
float j = 5.67.8;
```

1 Rappel : Les types primitifs en C sont : l'entier (`int`), le booléen (`bool`), le caractère (`char`) et le réel (`float` ou `double`)

`printf` et `scanf` sont les deux seules portes d'entrée et de sortie du programme. Le spécificateur de format doit correspondre exactement au type de la variable, sans quoi le compilateur laisse passer et le programme se trompe.

1.4. Soient les déclarations suivantes :

```
int a = 10;
float b = 5.565;
char c = 'A';
```

Corrige les lignes suivantes lorsque c'est nécessaire :

```
printf("Valeur de a: %d\n", a);
printf("Valeur de b: %d\n", b);
printf("Valeur de c: %c\n", c);
printf("Valeur de d: %f\n", d);
printf("Somme a+b: %c\n", a + b);
printf("Valeur absolue de -a: %d\n", a*-1);
printf("Valeur de c : %f\n", c);
printf("Produit a*d: %f\n", a * d);
printf("Valeur de b: %.2f\n", b);
printf("Valeur de b: %c\n", b);
```

1.5. Corrige le code suivant :

```
int a; float b; char c;
scanf("%d", a);
scanf("%f", &a);
scanf("%c", &b);
scanf("%d %d", &a, &b);
scanf("%s", &c);
```

Les opérateurs et les conversions

Rappel : en C, la division d'un entier par un entier donne un entier (et pas un réel).

1.6. Sans écrire de programme, prédire la valeur et le type de chaque expression :

Expression	*Type*	*Valeur*
`7 / 2`		
`7 % 2`		
`7.0 / 2`		
`5 / 2.0`		
`(float)7 / 2`		
`(float)(7 / 2)`		
`-7 / 2`		
`-7 % 2`		
`1 / 3 * 3`		
`(int)5.9`		
`(int)(5.9 + 0.5)`		
`9 / 2 / 2`		
`10 % 3 % 2`		
`2 + 3 * 4 % 5`		
`'A' + 1`		
`(char)('A' + 1)`		
`'z' - 'a'`		

1.7. Donner l'état de chaque variable après chaque instruction :

Instruction	*Valeur de a*	*Valeur de b*
`int a = 5, b = 2;`		
`a = a + b;`		
`b = a - b;`		
`a = a - b;`		
`a += 3;`		
`b *= a;`		

Instruction	*Valeur de a*	*Valeur de b*
`int a = 4, b = 0;`		
`b = a++;`		
`b = ++a;`		
`a -= b / 2;`		
`b %= 3;`		
`a *= b + 1;`		

Lire et déboguer un programme

Avant d'écrire un programme, il faut savoir lire celui des autres et repérer une erreur que le compilateur ne signalera pas.

1.8. Pour chaque programme, écris l'affichage exact produit (attention aux espaces, aux retours à la ligne et aux décimales). Si le programme comporte une erreur, signale-la et propose une correction.

Programme 1

```
int stock = 8;
printf("Stock :\t%d articles\nMerci de votre visite !", stock);
```

Programme 2

```
float temperature = 18.6;
printf("Il fait %d degres aujourd'hui.", temperature);
```

Programme 3

```
int heures = 9;
int minutes = 5;
printf("Rendez-vous a %dh%d le %d mai.", heures, minutes);
```

Programme 4

```
int jour = 3;
int mois = 7;
printf("Date d'echeance : %02d/%02d/2026", jour, mois);
```

Programme 5

```
char note = 'B';
int points = 78;
printf("Tu obtiens la note %s avec %d points.", note, points);
```

Programme 6

```
int addition = 7;
int nbPersonnes = 2;
float part = addition / nbPersonnes;
printf("Chacun paie %.2f euros.", part);
```

Programme 7

```
int prixArrondi = 19.99;
char devise = "E";
printf("Prix arrondi : %d %c", prixArrondi, devise);
```

Programme 8

```c
char rayon = 'F';
float prixKg = 12.4;
int quantite = 3;
printf("Rayon %c | %-8.2f | x%d", rayon, prixKg, quantite);
```

Programme 9

```c
char lettre = 'a';
printf("Lettre %c -> code %d -> majuscule %c", lettre, lettre, lettre - 32);
```

Programme 10

```c
int reduction = 30;
printf("Soldes : -%d% sur tout le magasin !", reduction);
```

Programme 11

```c
float tauxTVA = 0.21;
float prixHT = 100;
printf("Prix TTC : %.2f\n", prixHT * 1 + tauxTVA);
```

Programme 12

```c
int a = 5;
int b = 3;
printf("%d + %d = %d\n", a, b, a + b);
printf("%d x %d = %d\n", a, b, a * b);
printf("%d / %d = %d\n", a, b, a / b);
printf("%d %% %d = %d\n", a, b, a % b);
```

Programme 13

```c
char prenom = 'Tom';
int age = 16;
printf("%c a %d ans.", prenom, age);
```

Programme 14

```c
int nbEleves = 24;
int nbGroupes = 5;
printf("%d groupes de %d eleves, il en reste %d.",
       nbGroupes, nbEleves / nbGroupes, nbEleves % nbGroupes);
```

Programme 15

```c
float distance = 1234.5678;
printf("|%10.1f|\n", distance);
printf("|%-10.1f|\n", distance);
printf("|%.3f|\n", distance);
printf("|%f|\n", distance);
```

Programme 16

```c
int compteur = 4;
printf("Il reste %d essais.\n" compteur);
```

Programme 17

```c
char symbole = '#';
printf("%c%c%c\n", symbole, symbole, symbole);
printf("%3c|\n", symbole);
printf("Fichier : \"rapport.txt\"\n");
printf("Chemin : C:\\Documents\\Ecole");
```

Programme 18

```c
int total = 250;
int nbArticles = 4;
printf("Prix moyen : %.2f euros", total / nbArticles);
```

Programme 19

```c
int annee = 2026;
char classe = 5;
printf("Annee %d - classe %c", annee, classe);
```

Programme 20

```c
float note1 = 14.5;
float note2 = 11.25;
float moyenne = (note1 + note2) / 2;
printf("Note 1  : %5.2f\n", note1);
printf("Note 2  : %5.2f\n", note2);
printf("Moyenne : %5.2f", moyenne);
```

1.9. L'école organise un souper et propose un menu enfant (moins de 12ans) à 8€50 et un menu adulte à 14€. Écris un programme qui demande à l'utilisateur le nombre d'enfants qui seront présents ainsi que le nombre d'adultes. Le dialogue se présentera de la manière suivante :

```
Bonjour, merci de participer a notre souper.

Combien d'adultes prendront part au souper ?

> 1

Combien d'enfants prendront part au souper ?

> 1
```

Le programme affichera alors le prix à payer à la famille sous la forme suivante. Veillez à afficher une réponse en nombre flottants à deux décimales de précisions.

```
Prix a payer : 22.50 euro
```

1.10. Écris un programme qui demande deux entiers à l'utilisateur et qui affiche leur produit, leur somme, leur division ainsi que leur soustraction. *Pour quelle(s) valeur(s) le programme risque-t-il de renvoyer une erreur ?* Modifiez ensuite le programme afin de saisir deux réels.

1.11. Écris un programme qui échange deux entiers saisis. Ecris à l'écran avant et après l'échange :

`la variable X vaut : … la variable Y vaut : …`

Division entière et modulo

`/` donne le quotient, `%` donne le reste. À eux deux, ils suffisent à décomposer n'importe quelle grandeur (une durée, un montant, un nombre, …).

1.12. Écris un programme qui demande trois notes entières sur 100 et affiche leur moyenne avec deux décimales. Teste-le avec 70, 70 et 71 : le résultat est-il correct ? Corrige le programme si nécessaire et explique l'erreur.

1.13. Écris un programme qui demande une durée en secondes et l'affiche sous la forme heures / minutes / secondes.

```
Duree en secondes ?
> 7385
2 h 3 min 5 s
```

1.14. Écris un programme qui demande un montant en euros (nombre entier de centimes, par ex. 287) et affiche la décomposition en pièces de 2 €, 1 €, 50 c, 20 c, 10 c, 5 c, 2 c et 1 c, en utilisant le minimum de pièces. *Aucune condition ni boucle : uniquement des divisions entières et des modulos.*

```
Montant en centimes ?
> 287
2 euros : 1
1 euro  : 0
50 cent : 1
20 cent : 1
10 cent : 1
5 cent  : 1
2 cent  : 1
1 cent  : 0
```

1.15. Écris un programme qui demande un nombre entier de trois chiffres, puis affiche séparément son chiffre des centaines, des dizaines et des unités, ainsi que la somme de ces trois chiffres.

Formules et calculs

Attention aux priorités et aux conversions de type !!

1.16. Écris un programme qui demande une température en degrés Celsius et affiche l'équivalent en Fahrenheit (`F = C × 9/5 + 32`) puis en Kelvin (`K = C + 273.15`), avec une décimale.

1.17. Écris un programme qui demande le rayon d'un cercle et affiche son périmètre et son aire. La valeur de π est définie simplement par `3.14`.

1.18. Un magasin affiche des prix hors TVA. Écris un programme qui demande un prix HT et un taux de TVA en pourcentage, puis affiche le montant de la TVA et le prix TTC, tous deux à deux décimales

Le code ASCII

Rappel : tout caractère (`char`) est en réalité un entier (`int`) encodé selon le code ASCII.

1.19. Écris un programme qui demande à l'utilisateur de rentrer un caractère du clavier et qui affiche le code ASCII et hexadécimal de ce caractère de la manière suivante

```
caractère A           code ASCII = 65       hexadecimal = 41
```

1.20. Écris un programme qui demande une lettre majuscule et affiche la minuscule correspondante, en n'utilisant que de l'arithmétique sur le code ASCII (pas de fonction de `<ctype.h>`). Affiche également le caractère précédent et le caractère suivant dans la table ASCII.

2. Les structures conditionnelles

Du pseudo-code au C

2.1. Réécris en langage C les programmes suivants écrits en pseudo-code (il n'est pas nécessaire d'initialiser les variables, mais déclare-les avec le type qui convient).

Programme 1

```
┌── if (temperature > 30)
│ sortir "Alerte canicule !"
└──
```

Programme 2

```
┌── if (age ≥ 18 AND permis = 'O')
│ sortir "Vous pouvez louer une voiture."
└──
```

Programme 3

```
┌── if (solde < 0)
│ sortir "Compte a decouvert !"
├── else
│ sortir "Compte crediteur."
└──
```

Programme 4

```
┌── if (note ≥ 12)
│ sortir "Reussi"
├── else
│┌── if (note ≥ 10)
│| sortir "Deliberation"
│├── else
│| sortir "Echec"
│└──
└──
```

Programme 5

```
┌── if (stock = 0 OR fournisseur = 'X')
│ sortir "Produit indisponible"
└──
```

Programme 6

```
┌── if (NOT abonne)
│ sortir "Abonnez-vous pour continuer !"
├── else
│┌── if (nbArticlesLus > 50)
|| sortir "Lecteur assidu !"
│└──
└──
```

Programme 7

```
┌── if (vitesse > 120)
│ amende ← 150
│ sortir "Exces de vitesse grave"
├── else
│┌── if (vitesse > 90)
|| amende ← 50
|| sortir "Exces de vitesse leger"
│├── else
|| amende ← 0
|| sortir "Vitesse correcte"
│└──
└──
sortir "Montant de l'amende : " + amende
```

Programme 8

```
┌── if (categorie = 'S')
│┌── if (age < 12)
|| prix ← 5.0
│├── else
|| prix ← 8.5
│└──
├── else
│ prix ← 14.0
└──
sortir "Prix : " + prix + " euros"
```

Programme 9

```
┌── if (jour ≥ 1 AND jour ≤ 5)
│┌── if (heure < 12)
|| sortir "Cours du matin"
│├── else
|| sortir "Cours de l'apres-midi"
│└──
├── else
│┌── if (jour = 6 OR jour = 7)
|| sortir "Weekend"
│├── else
|| sortir "Jour invalide"
│└──
└──
```

Programme 10

```
┌── if (nbPoints ≥ 1000 AND anciennete > 2)
| statut ← 'G'
├── else
│┌── if (nbPoints ≥ 500 OR anciennete > 5)
|| statut ← 'S'
│├── else
|| statut ← 'B'
│└──
└──
sortir "Votre statut : " + statut
```

2.2. Écris un programme qui détermine si un entier saisi est pair ou impair

2.3. Écris un programme qui demande un nombre à l'utilisateur et l'informe ensuite si ce nombre est positif, négatif ou égal à zéro.

2.4. Ecris un programme qui demande deux nombres à l'utilisateur et l'informe ensuite si le produit est négatif, positif ou égal à zéro. Attention, à nouveau, on ne doit pas calculer le produit.

2.5. Écris un programme qui demande trois caractères à l'utilisateur puis qui informe ensuite s'ils sont rangés ou non dans l'ordre alphabétique. (On ne demande pas de restituer l'ordre alphabétique des trois caractères)

2.6. Écris un programme qui détermine le plus grand des trois entiers saisi.

If / Else en cascade

2.7. Un élève obtient une note sur 100. Le programme affiche : « Insuffisant » (< 50), « Satisfaisant » (50 à 64), « Bien » (65 à 79), « Tres bien » (80 à 89), « Excellent » (90 et plus).

Écris d'abord la version avec else if. Puis réécris-la en remplaçant tous les else if par des if indépendants : que se passe-t-il ?

2.8. Le prix d'un envoi dépend du poids :

- jusqu'à 50 g → 1,10 € ;
- jusqu'à 100 g → 1,80 € ;
- jusqu'à 350 g → 3,20 € ;
- jusqu'à 1 kg → 5,50 € ;
- au-delà → 8,90 €.

Le programme demande le poids en grammes et affiche le tarif.

2.9. Un magasin de reprographie facture 0,10€ les dix premières photocopies, 0,09€ les vingts suivantes et 0,08€ au-delà. Écrivez un algorithme qui demande à l'utilisateur le nombre de photocopies effectuées et qui affiche la facture correspondante.

2.10. L'IMC. L'utilisateur saisit son poids (kg) et sa taille (m). Le programme calcule l'IMC (poids / taille²) et affiche la catégorie correspondante selon les seuils de l'OMS : insuffisance pondérale (< 18,5), corpulence normale (18,5 à 25), surpoids (25 à 30), obésité (≥ 30). Le programme affichera également un rappel indiquant que ce calcul est indicatif et ne remplace pas un avis médical.

La structure Switch

2.11. Ecris un programme qui demande à l'utilisateur le jour de la semaine (saisie d'un nombre représentant le jour : lundi = 1 , mardi = 2, mercredi = 3...). Le programme restitue ensuite le planning du jour.
Pour ce faire, utilisez la structure alternative switch.

Pourquoi la structure SWITCH est-elle la plus adaptée dans ce contexte ?

Lundi	test Nlds – 18h : Rdv avec Julie
Mardi	Interrogation de math
Mercredi	14h30 : VTT avec Ben
Jeudi	Redaction de Francais a finir
Vendredi	Soiree Beauraing
Samedi	Repas Nadine
Dimanche	Repos

2.12. Écris un programme qui demande deux réels et un opérateur (+, -, *, /) puis affiche le résultat de l'opération. Utilise une structure switch. Attention : la division par zéro doit afficher un message d'erreur plutôt que de planter. Si l'opérateur n'est pas reconnu, indique « Operateur inconnu ».

```
Premier nombre : 12
Operateur (+ - * /) : /
Deuxieme nombre : 5
12.00 / 5.00 = 2.40
```

2.13. Écris un programme qui demande un numéro de mois (1-12) et affiche le nombre de jours qu'il contient. Pour février, on affichera 28. Utilise un switch en exploitant le fait que plusieurs case peuvent partager le même bloc d'instructions (absence de break).

Question de réflexion : combien de break ton programme contient-il ? Que se passerait-il si tu les supprimais tous ?

2.14. Même principe : l'utilisateur saisit un numéro de mois, le programme affiche la saison correspondante (par convention : décembre, janvier et février = hiver ; mars, avril, mai = printemps ; etc.).

2.15. Un distributeur propose : 1. Café (1,20 €), 2. Thé (1,00 €), 3. Chocolat (1,50 €), 4. Soupe (1,80 €). L'utilisateur choisit un numéro puis introduit une somme. Le programme affiche soit la monnaie à rendre, soit le montant manquant. Un choix invalide affiche « Selection invalide ».

Question de réflexion : pourquoi le switch convient-il ici pour le choix du produit mais pas pour la comparaison entre la somme introduite et le prix ?

2.16. L'utilisateur saisit un caractère. Le programme indique s'il s'agit d'une lettre majuscule, d'une lettre minuscule, d'un chiffre ou d'un autre symbole. Tu n'utiliseras que des comparaisons sur les codes ASCII, sans la bibliothèque `<ctype.h>`

2.17. Une année est bissextile si elle est divisible par 4, sauf si elle est divisible par 100 à moins qu'elle soit divisible par 400. Écris un programme qui détermine si une année saisie est bissextile.

Teste impérativement avec 2024, 2023, 1900 et 2000. Combien de conditions ton programme utilise-t-il ? Peux-tu l'écrire en une seule ligne de condition ?

2.18. Trois longueurs sont saisies. Le programme détermine d'abord si un triangle est constructible (chaque côté doit être inférieur à la somme des deux autres), puis, si oui, s'il est équilatéral, isocèle ou scalène.

2.19. L'utilisateur saisit un jour, un mois et une année. Le programme indique si la date est valide. Attention aux mois de 30 jours et à février en année bissextile.

2.20. Les habitants de Zorglub paient l'impôt selon les règles suivantes :

- Les hommes de plus de 20 ans paient l'impôt
- Les femmes paient l'impôt si elles ont entre 18 et 35 ans
- Les autres ne paient pas l'impôt

Le programme demandera donc l'âge et le sexe du Zorglubien (H/F) et se prononcera donc ensuite sur le fait que l'habitant est imposable ou non.

Les conditions imbriquées

2.21. Écris un programme qui demande le jour de la semaine (saisie d'un nombre représentant le jour : lundi = 1 , mardi = 2, mercredi = 3…) et la température.
S'il fait plus de 14° degrés mais moins de 25, le programme répondra « il fait bon aujourd'hui! », s'il fait plus chaud, le programme répondra « Il fait chaud ajd ! » sinon qu'il fait froid. Si nous sommes un jour de semaine, le programme souhaitera à l'utilisateur une bonne semaine ou sinon un bon weekend. Si le programme ne reconnaît pas l'encodage du jour, le programme indiquera « Je n'ai pas compris quel jour nous sommes, mais bonne journée ». Exemple de dialogue :

```
Quel jour de la semaine sommes-nous ? (LUNDI=1, MARDI=2, MERCREDI=3,
JEUDI=4, VENDREDI=5, SAMEDI=6, DIMANCHE=7)
> 1
Quelle est la température ?
> 23
Bonjour,
Il fait bon aujourd'hui. Bonne semaine !
```

2.22. Le feu tricolore. L'utilisateur saisit la couleur du feu (V, O, R) et sa distance en mètres par rapport au feu. Le programme indique s'il doit s'arrêter ou passer : au vert on passe ; au rouge on s'arrête ; à l'orange, on passe seulement si l'on est à moins de 10 mètres (freinage impossible), sinon on s'arrête.

2.23. Les frais de livraison. Une boutique en ligne applique les règles suivantes : livraison gratuite dès 50 € d'achat ; sinon 4,95 € en Belgique et 9,95 € à l'étranger ; les membres du programme fidélité bénéficient de la gratuité dès 30 €. Le programme demande le montant, le pays (B ou E) et si le client est membre, puis affiche les frais et le total.

2.24. Donne le maximum de trois entiers et affiche-les triés dans l'ordre croissant.

2.25. Pierre-feuille-ciseaux. Deux joueurs saisissent chacun leur choix (P, F, C). Le programme annonce le vainqueur ou l'égalité. Écris-le d'abord avec des if imbriqués, puis compare avec une version utilisant switch.

2.26. Une compagnie d'assurance automobile propose à ses clients quatre familles de tarifs identifiables par une couleur, du moins au plus onéreux : tarifs bleu, vert, orange et rouge. Le tarif dépend de la situation du conducteur :

- un conducteur de moins de 25 ans et titulaire du permis depuis moins de deux ans, se voit attribuer le tarif rouge, si toutefois il n'a jamais été responsable d'accident. Sinon, la compagnie refuse de l'assurer.
- un conducteur de moins de 25 ans et titulaire du permis depuis plus de deux ans, ou de plus de 25 ans mais titulaire du permis depuis moins de deux ans a le droit au tarif orange s'il n'a jamais provoqué d'accident, au tarif rouge pour un accident, sinon il est refusé.
- un conducteur de plus de 25 ans titulaire du permis depuis plus de deux ans bénéficie du tarif vert s'il n'est à l'origine d'aucun accident et du tarif orange pour un accident, du tarif rouge pour deux accidents, et refusé au-delà
- de plus, pour encourager la fidélité des clients acceptés, la compagnie propose un contrat de la couleur immédiatement la plus avantageuse s'il est entré dans la maison depuis plus de cinq ans. Ainsi, s'il satisfait à cette exigence, un client normalement "vert" devient "bleu", un client normalement "orange" devient "vert", et le "rouge" devient orange.

Écrire le programme permettant de saisir les données nécessaires (sans vérification de saisie) et de traiter ce problème.

2.27. **Dépassement.** L'utilisateur saisit les coefficients `a`, `b` et `c` de l'équation $ax^2 + bx + c = 0$. Le programme calcule le discriminant (le delta) et affiche le nombre de solutions ainsi que leur valeur. N'oublie pas le cas où a vaut 0 : l'équation n'est alors plus du second degré.

Tu auras besoin de la fonction `sqrt()` de `<math.h>`

3. Les structures itératives

Comprendre les structures itératives

3.1. Écris trois versions d'un même programme qui affiche les nombres de 1 à 10 : une avec while, une avec do…while, une avec for. Complète ensuite le tableau :

	`while`	`do while`	`for`
Le test a lieu…			
Nombre minimal d'exécutions du corps			
Le compteur est déclaré…			
Situation où elle est la plus adaptée			

3.2. Pour chaque situation, indique la structure la plus adaptée et justifie ton choix en une phrase.

Situation	*Structure*	*Pourquoi ?*
Afficher les 12 mois de l'année		
Demander un mot de passe jusqu'à ce qu'il soit correct		
Lire des notes jusqu'à la saisie d'un nombre négatif		
Calculer la somme des entiers de 1 à n		
Afficher un menu tant que l'utilisateur ne choisit pas « Quitter »		
Chercher un diviseur d'un nombre		
Afficher les 26 lettres de l'alphabet		

3.3. Pour chaque scénario, complétez les deux dernières colonnes en indiquant les conditions de continuation et d'arrêt correspondantes.

Problématique et propositions	Je m'arrête si...	Je continue tant que...
Recherche d'une personne dans une liste de contacts A = J'ai trouvé la personne B = J'ai atteint la fin de la liste		
Attente d'un événement important A = Il est l'heure prévue B = Un message d'urgence est reçu		
Surveillance d'un système informatique A = Le système est en panne B = Une maintenance est programmée		
Processus de validation d'un document A = Toutes les signatures sont présentes B = Le délai est expiré		
Contrôle de qualité d'un produit A = Le produit est conforme aux normes B = Des défauts mineurs sont acceptables		
Processus d'admission à une université A = La moyenne est supérieure à 14 B = L'entretien est réussi		
Diagnostic médical A = Le patient a de la fièvre B = Le patient présente des symptômes respiratoires		
Processus de recrutement A = Le candidat a les compétences techniques B = Le candidat a une bonne attitude		
Système d'alarme A = Une intrusion est détectée B = Le système est en mode test		

3.4. Donne la condition du while permettant de vérifier l'entrée de l'utilisateur sur base de la problématique donnée

Problématique	Condition du while
Demander à l'utilisateur d'entrer un chiffre entre 1 et 5.	
Demander à l'utilisateur d'entrer une lettre majuscule 'O' pour Oui ou 'N' pour Non.	
Demander à l'utilisateur d'entrer un nombre entre 10 et 20 inclus.	
Demander à l'utilisateur de choisir une option parmi 'A', 'B', 'C' ou 'D'.	
Demander à l'utilisateur d'entrer un nombre qui soit pair ou supérieur à 100.	
Demander à l'utilisateur d'entrer un caractère qui soit une lettre (majuscule ou minuscule).	
Demander à l'utilisateur d'entrer une température en Celsius qui soit soit négative, soit supérieure à 100.	
Demander à l'utilisateur d'entrer un nombre qui soit divisible par 3 ou par 5.	
Demander à l'utilisateur d'entrer un jour de la semaine (1-7).	
Demander à l'utilisateur d'entrer un caractère qui soit une voyelle (a, e, i, o, u, y).	
Demander à l'utilisateur d'entrer un code postal valide (entre 1000 et 9999 ou égal à 10000).	

3.5. Réécris chacune de ces boucles avec la structure demandée, sans modifier son comportement.

```c
// (a) → en for
int i = 10;
while (i > 0) {
    printf("%d ", i);
    i -= 2;
}

// (b) → en while
for (int i = 1, j = 20; i < j; i++, j--) {
    printf("%d-%d ", i, j);
}

// (c) → en do...while
int n;
scanf("%d", &n);
while (n < 0) {
    printf("Valeur invalide !\n");
    scanf("%d", &n);
}
```

Lire et déboguer une boucle

3.6. Sans utiliser l'ordinateur, écris l'affichage exact de chacun de ces programmes. Si une boucle ne se termine jamais, écris « BOUCLE INFINIE » et explique pourquoi.

```c
// (a)
for (int i = 0; i < 5; i++) {
    printf("%d ", i);
}
// (b)
for (int i = 0; i <= 5; i++) {
    printf("%d ", i);
}
// (c)
for (int i = 5; i > 0; i--) {
    printf("%d ", i);
}
// (d)
int i = 0;
while (i < 5) {
    printf("%d ", i);
}
// (e)
for (int i = 0; i < 5; i++);
    printf("%d ", i);
// (f)
for (int i = 0; i != 10; i += 3) {
    printf("%d ", i);
}
// (g)
int n = 5;
while (n > 0) {
    printf("%d ", n);
    n = n - 2;
}
// (h)
for (float x = 0; x != 1.0; x += 0.1) {
    printf("%.1f ", x);
}
```

3.7. Déroule ce programme à la main, une ligne par tour de boucle.

```c
int a = 1, b = 10;
while (a < b) {
    a = a * 2;
    b = b - 1;
    printf("%d %d\n", a, b);
}
```

Tour	a < b	Valeur de a	Valeur de b	Affichage
Init.				
1				
2				
3				
4				
5				

3.8. Sans exécuter, indique le nombre de tours de chaque boucle et la valeur du compteur après la boucle.

Boucle	Nombre de tours	Valeur finale
`for (int i = 0; i < 10; i++)`		
`for (int i = 1; i <= 10; i++)`		
`for (int i = 0; i < 10; i += 2)`		
`for (int i = 10; i > 0; i--)`		
`for (int i = 0; i < 0; i++)`		
`for (int i = 1; i <= 100; i *= 2)`		
`for (int i = 0; i <= n; i++)`		

3.9. Chacun de ces programmes contient un bug. Identifie-le, explique ses conséquences, puis corrige-le.

```c
// (a) Somme des 10 premiers entiers
int somme;
for (int i = 1; i <= 10; i++) {
    somme = somme + i;
}
printf("%d\n", somme);

// (b) Compte à rebours de 10 à 1
int i = 10;
while (i > 0) {
    printf("%d ", i);
    i++;
}

// (c) Moyenne de 5 notes
int total = 0;
for (int i = 0; i < 5; i++) {
    int note;
    scanf("%d", &note);
    total += note;
    printf("Moyenne : %.2f\n", (float)total / i);
}

// (d) Saisie jusqu'à obtenir un âge valide
int age;
do {
    printf("Age : ");
    scanf("%d", &age);
} while (age > 0 && age < 120);
```

3.10. Écrire un programme qui permette de saisir un nombre entier positif. Et ensuite, d'afficher la suite `1,2,3,4,…,n`

3.11. Écrire un algorithme qui demande un nombre de départ, et qui ensuite affiche les dix nombres suivants. Par exemple, si l'utilisateur entre le nombre 17, le programme affichera les nombres de 18 à 27.

3.12. Crée un programme qui affiche la table de multiplication d'un chiffre.

```
Entrez un chiffre :
> 3
La table de multiplication pour 3 :
1 * 3 = 3
2 * 3 = 6
3 * 3 = 9
…
10 * 3 = 30
```

3.13. Vous organisez une soirée. Les tickets boissons se présentent sous deux catégories : les tickets bleus à 1,80 euros et les tickets rouges à 2,20 euros. Créez un logiciel qui affiche la liste des conversion de 1 à 40 tickets.
Utilisez une structure itérative pour résoudre l'énoncé. Attention, portez une attention particulière à l'orthographe de votre tableau. Exemple du dialogue :

```
1 ticket bleu: 1.80          |     1 ticket rouge : 2.20
2 tickets bleu: 3.60         |     2 tickets rouge : 4.40
3 tickets bleu: 5.40         |     3 tickets rouge : 6.60
4 tickets bleu: 7.20         |     4 tickets rouge : 8.80
5 tickets bleu: 9.00         |     5 tickets rouge : 11.00
…
```

3.14. Écris un programme qui demande un nombre de départ et calcule la somme des entiers jusqu'à ce nombre. Par exemple, si l'on entre 5, le programme calcule 1 + 2 + 3 + 4 + 5 = 15.
NB : on souhaite afficher uniquement le résultat, pas la décomposition du calcul.

3.15. Crée un programme qui calcule la factorielle d'un nombre saisi au clavier. Quelle valeur maximale peut être calculée ? Que se passe-t-il au-delà ? Le programme signale-t-il l'erreur ?

3.16. Crée un programme qui demande à l'utilisateur de saisir une série de N entiers et qui affiche leur somme, leur produit et leur moyenne. Le nombre N est également saisi au clavier.

3.17. Écris un programme qui demande N nombres et affiche le plus grand.
Astuce : les nombres saisis ne sont pas tous conservés en mémoire. Avec quelle valeur initialiser la variable qui contiendra le maximum ? Teste ta réponse avec une série de nombres tous négatifs.

3.18. Écrire un algorithme qui demande à l'utilisateur un nombre compris entre 20 et 25 jusqu'à ce que la réponse convienne. Quelle structure as-tu utilisée ? Pourquoi le do…while est-il ici plus naturel que le while ?

3.19. Écris un programme qui pose une question et n'accepte que O ou N en réponse, en majuscules ou en minuscules. Toute autre saisie provoque un message d'erreur et une nouvelle demande.

```
Voulez-vous continuer ? (O/N)
> x
Reponse invalide, tapez O ou N.
> o
Tres bien, on continue !
```

3.20. Écris un programme qui demande un code PIN (un entier). Le code correct est 1234. L'utilisateur dispose de 3 tentatives. À chaque échec, le programme indique le nombre de tentatives restantes. Après trois échecs, il affiche « Carte bloquee ».
Ta boucle doit s'arrêter dans deux cas différents. Comment écrire cette condition ? Relis l'exercice 3.3.

3.21. Écris un programme qui calcule la somme, le produit et la moyenne d'une suite de nombres positifs non nuls entrés au clavier, sachant que la suite est terminée par zéro.
Bonus : si le nombre est négatif, indique à l'utilisateur un code d'erreur et propose-lui d'insérer un nouveau nombre.

3.22. Écris un programme qui affiche le plus grand et le plus petit d'une suite d'entiers saisis. La saisie se termine par l'encodage du zéro.
Astuce : les nombres saisis ne sont pas tous conservés en mémoire.

3.23. Écris un programme qui calcule la moyenne de notes fournies au clavier. Le nombre de notes n'est pas connu a priori. Pour signaler qu'il a terminé, l'utilisateur fournira une note fictive négative, qui ne devra naturellement pas être prise en compte dans le calcul.

```
note 1 : 12
note 2 : 15.25
note 3 : 13.5
note 4 : 8.75
note 5 : -1
moyenne de ces 4 notes : 12.37
```

3.24. Écris un programme qui lit une suite d'entiers terminée par zéro et affiche combien il y avait de nombres positifs, de nombres négatifs, ainsi que le pourcentage de chaque catégorie.
Que doit faire ton programme si l'utilisateur tape 0 dès la première saisie ?

```
Nombre (0 pour terminer) : 5
Nombre (0 pour terminer) : -3
Nombre (0 pour terminer) : 8
Nombre (0 pour terminer) : 0

3 nombres saisis
Positifs : 2 (66.67 %)
Negatifs : 1 (33.33 %)
```

3.25. Écris un programme qui affiche la somme des chiffres d'un entier positif saisi. Exemple : 4527 → 18.

3.26. Écris un programme qui compte le nombre de chiffres d'un entier positif. Exemple : 4527 → 4. Piège : que renvoie ton programme pour le nombre 0 ?

3.27. Écris un programme qui affiche un entier à l'envers. Exemple : 1234 → 4321.

3.28. Écris un programme qui détermine si un nombre est un palindrome (7227, 1331, 5…). Réutilise le principe de l'exercice précédent.

3.29. Écris un programme qui convertit un entier positif en binaire. Exemple : 13 → 1101.
Astuce : divise successivement par 2 en conservant les restes. Dans quel ordre apparaissent-ils par rapport à l'affichage souhaité ? Comment résoudre ce problème sans tableau ?

3.30. Afficher un triangle rempli d'étoiles, s'étendant sur un nombre de lignes fourni en donnée et se présentant comme dans cet exemple :

```
*
**
***
****
*****
```

3.31. Reprends le programme précédent et adapte-le pour produire chacune de ces figures, pour un nombre de lignes fourni par l'utilisateur.

```
(a) Triangle inversé     (b) Aligné à droite     (c) Pyramide       (d) Sablier
*****                          *                     *             *****
****                          **                    ***             ***
***                           ***                  *****             *
**                            ****                *******           ***
*                             *****              *********         *****
```

Pour (b) et (c), il te faudra deux boucles internes : une pour les espaces, une pour les étoiles.

3.32. Écris un programme qui affiche les tables de multiplication des nombres de 1 à 10, sous la forme suivante :

```
    |   1   2   3   4   5   6   7   8   9  10
----+----------------------------------------
  1 |   1   2   3   4   5   6   7   8   9  10
  2 |   2   4   6   8  10  12  14  16  18  20
3 |   3   6   9  …
```

3.33. La suite de Fibonacci est une suite d'entiers dans laquelle chaque terme est la somme des deux termes qui le précèdent. Elle commence par les termes 0 et 1 et ses premiers termes sont :

F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	...	F_n
0	1	1	2	3	5	8	13		$F_{n-1}+F_{n-2}$

Crée un programme qui calcule la valeur du $n^{ième}$ terme de la suite. Cette fonction sera codée de manière itérative.

3.34. Écris un programme qui calcule le plus grand commun diviseur de deux entiers par l'algorithme d'Euclide : tant que `b` n'est pas nul, on remplace le couple `(a, b)` par `(b, a % b)`.
Vérifie avec 48 et 18 (résultat attendu : 6).

3.35. Calcule b^n à l'aide d'une boucle, sans utiliser `pow()`.
Que se passe-t-il si n vaut 0 ? Si n est négatif ? Adapte ton programme en conséquence.

3.36. En utilisant la structure itérative, imagine un système de résolution pour détecter si un chiffre est premier ou non. Expliquez en français, comment il serait possible de résoudre le problème.

- Commencez par définir un nombre premier.

- Ensuite, que doit-on tester sur ce nombre pour vérifier s'il est premier ?

Dans le programme, l'utilisateur insère un nombre. Le programme teste s'il est premier ou pas. Le programme écrit « le nombre inséré est premier » ou « le nombre inséré n'est pas premier ». Imagine un pseudo code qui résolve ce problème puis implémente ce code en C.

3.37. Un nombre est parfait si la somme de ses diviseurs stricts vaut le nombre lui-même (6 = 1 + 2 + 3). Affiche tous les nombres parfaits inférieurs à 10 000.

3.38. Pars d'un entier positif. S'il est pair, divise-le par 2 ; sinon, multiplie-le par 3 et ajoute 1. Répète jusqu'à atteindre 1. Affiche la suite complète ainsi que le nombre d'étapes.
Teste avec 27 : combien d'étapes ? Quelle est la valeur maximale atteinte ?

Personne n'a jamais démontré que cette suite (La suite de Syracuse) atteint toujours 1, quel que soit le nombre de départ. Ton programme pourrait donc théoriquement ne jamais s'arrêter. C'est l'une des plus célèbres conjectures non résolues des mathématiques.

3.39. On te demande d'écrire un programme qui simule un combat entre un joueur et un ennemi.

Le programme doit :

- Initialiser les points de vie : le joueur commence avec 100 points de vie (PV) et l'ennemi commence avec 80 points de vie (PV)
- À chaque tour :
  - Demander au joueur de choisir une action : Attaque normale (inflige entre 10 et 20 dégâts) ou Attaque puissante (inflige entre 20 et 35 dégâts)
  - Générer les dégâts aléatoirement selon l'attaque choisie
  - Afficher les dégâts infligés et les PV restants de l'ennemi
  - Si l'ennemi est encore en vie, il contre-attaque automatiquement (inflige entre 8 et 18 dégâts)
  - Afficher les dégâts subis et les PV restants du joueur
  - Le combat s'arrête lorsqu'un des deux personnages n'a plus de PV
- À la fin du combat :
  - Si les PV du joueur sont supérieurs à 0 : afficher "Victoire !"
  - Sinon : afficher "Défaite !"

Un exemple d'exécution se trouve à la page suivante.

Voici un bout de code permettant de générer un nombre aléatoire entre 0 et 100 (stocké dans la variable `random`) :

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void main(void) {
    srand(time(0));
    int random;
    random = rand() % 101;
}
```

```
=== COMBAT ===
Joueur : 100 PV | Ennemi : 80 PV

--- Tour 1 ---
Choisissez votre action :
1. Attaque normale (10-20 dégâts)
2. Attaque puissante (20-35 dégâts)
> 2
Vous infligez 28 dégâts !
Ennemi : 52 PV restants

L'ennemi contre-attaque et inflige 15 dégâts !
Joueur : 85 PV restants

--- Tour 2 ---
Choisissez votre action :
1. Attaque normale (10-20 dégâts)
2. Attaque puissante (20-35 dégâts)
> 1
Vous infligez 17 dégâts !
Ennemi : 35 PV restants

L'ennemi contre-attaque et inflige 12 dégâts !
Joueur : 73 PV restants

[... le combat continue ...]

--- Fin du combat ---
Victoire !
```

3.40. Écris un programme en C qui simule un retrait d'argent à un distributeur automatique.

- Le programme doit dans un premier temps :
    - Initialiser le solde du compte du compte à 500 euros
    - Demander à l'utilisateur le montant qu'il souhaite retirer
- À chaque tentative où le montant est trop élevé :
    - Afficher "Solde insuffisant ! Il vous reste X euros."
    - Afficher "Tentative Y sur 3"
    - Redemander un nouveau montant
- Le programme s'arrête lorsque le solde est suffisant ou que l'utilisateur n'a plus de tentative (3 maximum)
    - Si le montant est valide : soustraire le montant du solde et afficher "Retrait effectué ! Nouveau solde : X euros »
    - Si toutes les tentatives sont épuisées : afficher "Nombre maximum de tentatives atteint. Carte bloquée !"

Un exemple d'exécution se trouve à la page suivante.

```
=== DISTRIBUTEUR AUTOMATIQUE ===
Solde disponible : 500 euros

Montant à retirer : 600
Solde insuffisant ! Il vous reste 500 euros.
Tentative 1 sur 3

Montant à retirer : 550
Solde insuffisant ! Il vous reste 500 euros.
Tentative 2 sur 3

Montant à retirer : 200
Retrait effectué ! Nouveau solde : 300 euros
```

```
=== DISTRIBUTEUR AUTOMATIQUE ===
Montant à retirer : 600
Solde insuffisant ! Il vous reste 500 euros.
Tentative 1 sur 3

Montant à retirer : 700
Solde insuffisant ! Il vous reste 500 euros.
Tentative 2 sur 3

Montant à retirer : 650
Solde insuffisant ! Il vous reste 500 euros.
Tentative 3 sur 3

Nombre maximum de tentatives atteint. Carte bloquée !
```

3.41. L'ordinateur tire un nombre au hasard entre 1 et 100. L'utilisateur propose des nombres ; le programme répond « c'est plus grand » ou « c'est plus petit » jusqu'à ce qu'il trouve, puis affiche le nombre de tentatives.

Prolongements :

- Limite le nombre de tentatives à 10. L'utilisateur perd au-delà.
- Quelle est la stratégie optimale ? En combien de coups au maximum peut-on toujours gagner ? Justifie ta réponse.

3.42. Affiche le triangle de Pascal jusqu'à la ligne N, sans utiliser de tableau.

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
```

Astuce : chaque coefficient se déduit du précédent de la même ligne par une multiplication et une division.

4. Les chaînes de caractères

Comprendre ce qu'est une chaîne

Rappel. Le C ne possède pas de type « chaîne de caractères ». Une chaîne est un tableau de `char` terminé par le caractère `'\0'` (code ASCII 0). Ce caractère invisible n'est pas décoratif : c'est lui, et lui seul, qui indique à printf et à toutes les fonctions de `<string.h>` où la chaîne s'arrête.

4.1. Complète ce tableau. Il te servira de référence pour tout le chapitre.

Fonction	*Rôle*	*Valeur renvoyée*
`strlen(s)`		
`strcpy(dest, src)`		
`strcat(dest, src)`		
`strcmp(s1, s2)`		

4.2. Pour chaque écriture, indique de quoi il s'agit, le type correspondant et la place occupée en mémoire.

Ecriture	*Caractère / Chaine ?*	*Type*	*Octets occupés*
`'A'`			
`"A"`			
`'\0'`			
`"0"`			
`'0'`			
`""`			
`"Bonjour"`			
`'AB'`			

4.3. Complète le tableau pour chacune de ces déclarations.

```
char a[10] = "Salut";
char b[]   = "Salut";
char c[5]  = "Salut";
char d[6]  = {'S', 'a', 'l', 'u', 't', '\0'};
char e[5]  = {'S', 'a', 'l', 'u', 't'};
char f[20] = "";
```

Variable	*Taille réservée (sizeof)*	*Caractères utiles (strlen)*	*Position du '\0'*	*Déclaration valide ?*
a				
b				
c				
d				
e				
f				

4.4. Lesquelles des chaînes suivantes sont initialisées correctement ? Corrige les déclarations fausses et indique pour chacune le nombre d'octets réservés en mémoire.

```
char a[]   = "un\ndeux\ntrois\n";
char b[12] = "un deux trois";
char c[]   = 'abcdefg';
char d[10] = 'x';
char e[5]  = "cinq";
char g[2]  = {'a', '\0'};
char h[4]  = {'a', 'b', 'c'};
char i[4]  = "'o'";
```

4.5. Prédis l'affichage de ce programme, puis vérifie sur machine.

```
char mot[20] = "Bonjour";
printf("%d\n", sizeof(mot));
printf("%d\n", strlen(mot));
printf("%d\n", sizeof("Bonjour"));
printf("%d\n", strlen("Bonjour"));
```

4.6. Sans machine, écris l'affichage exact de chaque programme. Si le programme est incorrect, écris « ERREUR » et explique.

```c
// (a)
char mot[] = "Ecole";
printf("%s\n", mot);
printf("%c\n", mot[0]);
printf("%c\n", mot[4]);
printf("%d\n", mot[0]);

// (b)
char mot[] = "Ecole";
printf("%s\n", mot + 2);

// (c)
char mot[] = "Ecole";
mot[3] = '\0';
printf("%s\n", mot);
printf("%d\n", strlen(mot));
printf("%d\n", sizeof(mot));

// (d)
char mot[10] = "Bonjour";
printf("%c\n", mot[8]);

// (e)
char mot[] = "Ecole";
printf("%s\n", mot[0]);

// (f)
char a[] = "abc";
char b[] = "abd";
printf("%d\n", strcmp(a, b));
printf("%d\n", strcmp(b, a));
printf("%d\n", strcmp(a, a));
```

4.7. Chacun de ces programmes contient au moins un bug. Identifie-le, explique-le, corrige-le.

```c
// (a) Copier un prénom
char prenom[20];
prenom = "Julie";
printf("%s\n", prenom);

// (b) Comparer deux mots
char mdp[20] = "secret";
char saisie[20];
scanf("%s", saisie);
if (saisie == mdp) {
    printf("Acces autorise\n");
}

// (c) Concaténer
char nom[10] = "Dubois";
char prenom[10] = "Jean";
strcat(nom, prenom);
printf("%s\n", nom);

// (d) Afficher la longueur
char mot[50];
printf("Un mot : ");
scanf("%s", &mot);
printf("Longueur : %d\n", strlen(mot));

// (e) Initialiser
char ville[15];
printf("%s\n", ville);
```

4.8. Compile et exécute ce programme, puis saisis exactement `Jean Dupont`.

```
char nom[50];
printf("Votre nom complet : ");
scanf("%s", nom);
printf("Bonjour %s !\n", nom);
```

(a) Qu'affiche le programme ? Où est passé le reste ?

(b) Remplace la saisie par `fgets(nom, 50, stdin);` et recommence. Quelle différence ?

(c) Complète le tableau :

	`scanf("%s",…)`	`fgets(…)`
S'arrête à...		
Limite la taille saisie ?		
Conserve le `\n` final ?		
Nécessite le & ?		

(d) Dans quel cas `scanf("%s", …)` reste-t-il acceptable ?

4.9. Ce programme se comporte bizarrement. Exécute-le, décris le problème, puis corrige-le.

```
char nom[50];
int age;
printf("Votre nom : ");
fgets(nom, 50, stdin);
printf("Votre age : ");
scanf("%d", &age);
printf("Bonjour %s, vous avez %d ans.\n", nom, age);
printf("Longueur du nom : %d\n", strlen(nom));
```

(a) Pourquoi le reste du message se retrouve-t-il à la ligne suivante ?

(b) La longueur affichée est-elle celle attendue ? Pourquoi ?

(c) Corrige en supprimant le `\n` final :

```
nom[strlen(nom) - 1] = '\0';
```

(d) Explique cette ligne mot à mot. Que se passerait-il si l'utilisateur validait sans rien saisir ?

4.10. L'école organise un souper et propose un menu enfant (moins de 12 ans) à 8,50 € et un menu adulte à 14 €. Écris un programme qui demande le nom de famille (les noms composés doivent pouvoir être encodés, max 128 caractères), le nombre d'enfants et le nombre d'adultes.

```
Bonjour, merci de participer a notre souper.
Quel est votre nom de famille ?
> Van der Berg
Combien d'adultes prendront part au souper ?
> 1
Combien d'enfants prendront part au souper ?
> 1
```

Le programme affichera alors le prix à payer à la famille sous la forme suivante. Veillez à afficher une réponse en nombre flottants à deux décimales de précisions.

```
Reservation : Van der Berg
Prix a payer : 22.50 euro
```

4.11. Écris un programme qui demande successivement le prénom, le nom, la ville et l'âge de l'utilisateur, puis affiche une fiche formatée :

```
+--------------------------------+
| Nom     : Jean Dubois          |
| Ville   : Beauraing            |
| Age     : 17 ans               |
+--------------------------------+
```

Les prénoms et villes composés doivent être acceptés. Soigne l'alignement des colonnes : quel spécificateur de format utilises-tu ? (Indice : `%-20s`.)

4.12. Écris un programme qui affiche la taille d'une chaîne de caractères donnée par l'utilisateur. Par exemple, pour « langage », le programme affichera 7.

(a) Le programme fonctionne-t-il sur une chaîne vide ? Sur une chaîne contenant des espaces ?

(b) Compare la valeur affichée avec `sizeof` du tableau. Pourquoi diffèrent-elles ?

4.13. Réponds aux questions

(a) Pourquoi l'instruction suivante est-elle refusée par le compilateur ?

```
char a[20], b[20] = "Bonjour";
a = b;
```

(b) Écris le programme qui copie correctement b dans a puis affiche les deux chaînes.

(c) Modifie ensuite a[0] en 'C'. Que devient b ? Explique.

(d) Que se passe-t-il si la destination est trop petite ? Le programme compile-t-il ? S'exécute-t-il ? Que fait-il exactement ?

```
char petit[5];
strcpy(petit, "Une chaine beaucoup trop longue");
```

4.14. Traduis le diagramme d'action suivant en langage C :

```
┌── *
│ Obtenir montantPaye
│┌── if (montantPaye = 0)
|| abonnement = "gratuit"
│├── else
||┌── if (montantPaye = 5)
||| abonnement = "basique"
||├── else
||| abonnement = "premium"
||└──
|└──
│ Sortir "L'abonnement actuel : " + abonnement
└──
```

Attention : `abonnement = "gratuit"` n'est pas traduisible littéralement en C. Quelle taille réserver pour la variable abonnement ?

4.15. Écris un programme qui demande un prénom et un nom, puis construit et affiche le nom complet sous la forme `Prenom NOM`.

(a) Version avec `strcat`. Attention : d'où vient l'espace entre les deux mots ?

(b) Quelle taille faut-il réserver pour la chaîne résultat ? Que se passe-t-il si elle est trop petite ?

(c) Pourquoi la destination d'un `strcat` doit-elle obligatoirement contenir déjà une chaîne valide (éventuellement vide) ? Teste :

```c
char resultat[50]; // non initialisé
strcat(resultat, "Bonjour");
```

4.16. Observe le programme suivant puis complète le tableau. Chaque ligne s'exécute à la suite de la précédente.

```
char a[20] = "chat";
char b[20] = "chien";
char c[20] = "";
```

Instruction	*Valeur de a*	*Valeur de b*	*Valeur de c*	*Valeur renvoyée*
`strlen(a)`				
`strcpy(c, a)`				
`strcat(c, b)`				
`strlen(c)`				
`strcpy(a, b)`				

Comparer

Rappel. `if (a == b)` compile sans erreur mais ne compare pas le contenu des chaînes.

4.17. Exécute ce programme :

```
char a[10] = "chat";
char b[10] = "chat";
printf("a == b ?  %d\n", a == b);
printf("strcmp ?  %d\n", strcmp(a, b));
printf("adresse de a : %p\n", a);
printf("adresse de b : %p\n", b);
```

(a) Les deux chaînes contiennent le même texte. Pourquoi `a == b` est-il faux ?

(b) Que compare exactement l'opérateur == appliqué à deux tableaux ?

(c) Formule la règle en une phrase.

(d) Une comparaison réussie avec `strcmp` renvoie `0`, c'est-à-dire faux. Quelle conséquence sur l'écriture du `if` ?

4.18. Prédis la valeur renvoyée par chaque appel : négative, nulle ou positive ?

Appel	*Signe du résultat*	*Pourquoi ?*
`strcmp("abc", "abd")`		
`strcmp("abd", "abc")`		
`strcmp("abc", "abc")`		
`strcmp("abc", "abcd")`		
`strcmp("Zebre", "abeille")`		
`strcmp("chat", "Chat")`		
`strcmp("", "a")`		

(a) Sur quoi `strcmp` se fonde-t-il exactement pour décider ? (Indice : relis la table ASCII.)

(b) Pourquoi "Zebre" vient-il avant "abeille" ? Est-ce l'ordre alphabétique attendu par un humain ?

(c) Comment contourner ce problème sans modifier les chaînes ?

4.19. Écris un programme qui demande à l'utilisateur de deviner un mot magique. Si la réponse est correcte, affiche « Bravo ! Vous avez trouvé le mot magique ! ». Sinon, affiche « Ce n'est pas ça, réessayez ! ».
Améliorations : ajouter plusieurs mots magiques, laisser 3 tentatives à l'utilisateur et boucler jusqu'à ce que le mot soit trouvé.

4.20. Écris un programme qui demande trois prénoms à l'utilisateur puis l'informe s'ils sont rangés ou non dans l'ordre alphabétique. On ne demande pas de restituer l'ordre alphabétique des trois prénoms.

4.21. Écris un programme qui demande le nom de l'utilisateur, le jour de la semaine et la température. Le programme souhaite la bienvenue à l'utilisateur. S'il fait plus de 14 °C mais moins de 25, le programme répond « Il fait bon aujourd'hui ! » ; s'il fait plus chaud, « Il fait chaud ajd ! » ; sinon, qu'il fait froid. Si nous sommes un jour de semaine, le programme souhaite une bonne semaine, sinon un bon weekend. Les jours sont saisis en plein texte, en majuscules. Si le programme ne reconnaît pas l'encodage, il indique « Je n'ai pas compris quel jour nous sommes, mais bonne journée ».

```
Quel est ton prenom ?
> Tom
Quel jour de la semaine sommes-nous ? (LUNDI, MARDI, MERCREDI, JEUDI,
VENDREDI, SAMEDI, DIMANCHE)
> MARDI
Quelle est la temperature ?
> 23
Bonjour Tom,
Il fait bon aujourd'hui. Bonne semaine !
```

Remarque : pourquoi ne peut-on pas utiliser un switch ici ?

4.22. Crée un programme qui détermine le mot le plus long parmi une liste de mots donnés par l'utilisateur. L'encodage se termine par « Q ».

```
Mot (Q pour terminer) : ordinateur
Mot (Q pour terminer) : clavier
Mot (Q pour terminer) : souris
Mot (Q pour terminer) : Q
Le mot le plus long est : ordinateur (10 lettres)
```

Prolongements :

(a) Affiche également le mot le plus court.

(b) Que fait ton programme si l'utilisateur tape `Q` dès le premier mot ?

(c) Q est la sentinelle : comment saisir malgré tout le mot `Q` comme donnée ?

4.23. Une école attribue à chaque élève un identifiant construit ainsi : première lettre du prénom + nom + année d'inscription.

```
Prenom : Jean
Nom : Dubois
Annee : 2026
Identifiant : jdubois2026
```

(a) Construis l'identifiant dans une variable identifiant à l'aide de strcpy et strcat, puis affiche-le. Indice : pour placer un seul caractère en début de chaîne, passe par une chaîne de deux cases :

```
char temp[2];
temp[0] = prenom[0];
temp[1] = '\0';
strcpy(identifiant, temp);
```

(b) L'année est un entier, `strcat` attend une chaîne. Comment résoudre le problème ? (Piste : saisir l'année directement sous forme de chaîne.)

(c) Limite la partie « nom » aux 5 premiers caractères. Une seule instruction suffit :

```
nom[5] = '\0';
```

Explique pourquoi cela fonctionne. Que devient le reste du tableau en mémoire ?

(d) Que se passe-t-il si le nom compte moins de 5 lettres ? Ton instruction du (c) est-elle encore correcte ? Corrige-la à l'aide de `strlen`.

(e) Vérifie que l'identifiant obtenu ne dépasse pas 20 caractères. Quelle taille as-tu réservée pour la variable identifiant ?

4.24. Écris un programme qui guide l'utilisateur à travers un formulaire d'inscription complet. Chaque champ doit être validé avant de passer au suivant.

```
=== INSCRIPTION ===
Prenom :
> Jean
Nom de famille :
> Van der Berg
Identifiant souhaite :
> jv
Trop court : 4 caracteres minimum.
Identifiant souhaite :
> jvanderberg
Mot de passe :
> secret
Trop court : 8 caracteres minimum.
Mot de passe :
> monsecret123
Confirmez le mot de passe :
> monsecret124
Les deux mots de passe ne correspondent pas.
Confirmez le mot de passe :
> monsecret123
Confirmer l'inscription ? (OUI/NON) :
> OUI
=== INSCRIPTION VALIDEE ===
Bienvenue Jean Van der Berg !
Identifiant : jvanderberg
```

Règles à implémenter :

- Le prénom et le nom acceptent les espaces (noms composés).
- L'identifiant compte entre 4 et 15 caractères. Il est redemandé tant qu'il ne convient pas.
- Le mot de passe compte au moins 8 caractères, et doit être saisi deux fois à l'identique.
- Le mot de passe ne peut pas être identique à l'identifiant.
- La confirmation finale n'accepte que OUI ou NON. Si l'utilisateur répond NON, le programme affiche « Inscription annulee ».
- Le message de bienvenue affiche le nom complet, construit dans une seule variable.

5. Les tableaux

5.1. Déclare un tableau d'entiers de taille 5, initialise-le avec les valeurs `[1,2,3,4,5]` et affiche chacun de ses éléments à l'aide d'une boucle.

5.2. Déclare un tableau de réels contenant les valeurs `[2.5,3.5,4.5,5.5]`, puis calcule et affiche la somme de tous ses éléments.

5.3. Déclare un tableau de caractères contenant la chaîne `"abracadabra"`. Affiche chaque caractère de la chaîne, puis détermine et affiche le caractère le plus fréquent dans le tableau.

5.4. Déclare un tableau de booléens de taille 5, initialisé avec `[true,false,true,false,true]`. Affiche les valeurs du tableau et compte combien de fois la valeur `true` apparaît.

5.5. Ecris un programme qui demande à l'utilisateur une série de prix-quantités puis qui affiche le résultat sous la forme d'un ticket de caisse (avec le total à la fin). L'encodage se termine lorsque l'utilisateur entre une quantité négative.

5.6. Ecris un programme qui demande à l'utilisateur deux chaines de caractères puis qui concatène la première avec la deuxième. Attention, tu ne peux pas utiliser la fonction `strcat` ni concaténer directement dans l'affichage du `printf`.

5.7. Ecris un programme qui demande à l'utilisateur une série d'entiers (l'encodage se termine lorsque l'utilisateur entre une valeur négative) puis qui affiche cette série à l'envers.

5.8. Ecris un programme qui vérifie si un tableau d'entiers est symétrique.

5.9. Crée un programme qui détermine si un mot est un palindrome ou non.

5.10. Ecris un programme qui demande à l'utilisateur d'insérer 10 entiers puis qui affiche la liste de ces entiers triés dans l'ordre croissant. Implémente l'algorithme de tri à bulles pour trier le tableau.

5.11. Écris un programme qui "compresse" un tableau de 0 et de 1 en déplaçant tous les 1 à gauche et tous les 0 à droite. Par exemple, le tableau `[1,0,1,0,0,1,0]` deviendrait `[1,1,1,0,0,0,0]`.

5.12. Ecris un programme qui demande une chaine de caractères à l'utilisateur puis qui remplace toutes les voyelles de cette chaine par le symbole `*`.

5.13. Ecris un programme qui demande une chaine de caractères à l'utilisateur ainsi qu'une lettre. Le programme doit se charger de remplacer toutes les occurrences de cette lettre dans la chaine par le symbole `*`.

5.14. Pour chaque problématique ci-dessous, complète la condition du while qui permettra de réaliser la recherche demandée.

Problématique	Condition
Vous avez un tableau d'entiers `nombres[]` de taille `taille`. Écrivez la condition d'une boucle while qui recherche la première occurrence de la valeur `valeurRecherchee` dans ce tableau.	
Vous avez un tableau d'identifiants `ids[]` de taille `nbIds`. Écrivez la condition d'une boucle while qui recherche l'identifiant `idRecherche` dans ce tableau.	
Vous avez un tableau d'entiers `nombres[]` de taille `taille`. Écrivez la condition d'une boucle while qui recherche le premier nombre négatif dans ce tableau.	
Vous avez un tableau d'entiers `nombres[]` de taille `taille`. Écrivez la condition d'une boucle while qui recherche le premier nombre pair dans ce tableau.	
Vous avez un tableau de températures `temperatures[]` de taille `nbTemperatures`. Écrivez la condition d'une boucle while qui recherche la première température supérieure à `seuil`.	
Vous avez une chaîne de caractères `texte[]`. Écrivez la condition d'une boucle while qui recherche la première occurrence du caractère `caractereRecherche` dans cette chaîne.	
Vous avez un tableau de chaînes de caractères `noms[]` contenant `nbNoms` éléments. Écrivez la condition d'une boucle while qui recherche la chaîne `nomRecherche` dans ce tableau.	
Vous avez un tableau de mots `mots[]` contenant `nbMots` éléments. Écrivez la condition d'une boucle while qui recherche le premier mot commençant par la lettre `lettre`.	

5.15. Écris un programme qui demande à l'utilisateur de saisir une série de nombres distincts (limités à une taille maximale de 10). Ensuite, demande-lui d'ajouter un nouveau nombre. Si ce nombre existe déjà dans le tableau, affiche un message d'erreur. Sinon, ajoute-le au tableau et affiche la liste mise à jour.

5.16. Écris un programme qui demande à l'utilisateur d'entrer 5 nombres distincts, puis un nombre à rechercher, et affiche sa position dans le tableau ou le message « Nombre introuvable » s'il n'est pas présent.

5.17. Écris un programme qui demande à l'utilisateur de saisir une série de nombres entiers. L'encodage s'arrête dès que l'utilisateur entre un nombre déjà saisi auparavant. Le programme doit ensuite afficher tous les nombres uniques saisis dans l'ordre d'entrée.

5.18. Écris un programme qui calcule et affiche la somme des éléments des deux diagonales d'une matrice carrée donnée par l'utilisateur. L'utilisateur donne dans un premier temps la taille `N` de la matrice (`N`x`N`) puis le contenu de la matrice sous la forme d'une suite d'entiers.

5.19. Ecris un programme qui demande à l'utilisateur d'entrer une chaîne de caractères et compresse cette chaîne en utilisant l'algorithme de compression RLE (Run-Length Encoding). Par exemple, la chaîne `"aaabbccccd"` deviendrait `"a3b2c4d1"`.

5.20. Ecris un programme qui demande à l'utilisateur de saisir une chaîne de chiffres (par exemple `"12345"`) puis qui convertit chaque caractère de la chaîne en un entier et qui stocke ces entiers dans un tableau.

5.21. Ecris un programme qui affiche la fréquence de chaque lettre dans une chaine de caractères entrée par l'utilisateur.

5.22. Écris un programme qui demande à l'utilisateur de saisir deux tableaux d'entiers triés (de tailles 5 chacun). Ensuite, fusionne ces deux tableaux dans un seul tableau trié en ordre croissant et affiche le résultat.

6. Les fonctions

6.1. Pour chaque programme, identifie les prototypes, les appels ainsi que les déclarations de fonctions.

```c
void f1(char p[]);

void main(void){
    f1("Texte");
}

void f1(char p[]) {
    printf("%s\n", p);
}
```

```c
int f2(int x, int y);

void main(void) {
    int r;
    r = f2(8, 3);
    printf("Résultat : %d\n", r);
}

int f2(int x, int y) {
    return x + y;
}
```

```c
void f3(void);
double f4(double a, double b);

void main(void){
    double r;
    f3();
    r = f4(2.1, 3.9);
    printf("Résultat : %.2f\n", r);
}

void f3(void) {
    printf("Message !\n");
}

double f4(double a, double b) {
    return a * b;
}
```

6.2. Pour chaque programme, identifie les paramètres effectifs et les paramètres formels.

```c
void afficherMessage(char message[]) {
    printf("Message: %s\n", message);
}

void main(void) {
    afficherMessage("Bonjour!");
}
```

```c
int somme(int a, int b) {
    return a + b;
}

void main(void) {
    int resultat = somme(10, 20);
    printf("Résultat: %d\n", resultat);
}
```

```c
void afficherDivision(int a, int b) {
    if (b != 0) {
        printf("Division: %d\n", a / b);
    } else {
        printf("Division par zéro impossible\n");
    }
}

void main(void) {
    afficherDivision(15, 3);
    afficherDivision(10, 0);
}
```

6.3. Pour chaque programme, détermine la portée (*scope*) de chaque variable.

```c
void f(int x, int y) {
    x = x + 10;
    y = y + 20;
    printf("x = %d, y = %d\n", x, y);
}

void main(void) {
    int a = 5, b = 15;
    f(a, b);
    printf("a = %d, b = %d\n", a, b);
}
```

La variable	est accessible de la ligne...	à la ligne...
x	2	4
y		
a		
b		

```c
void f() {
    int x;
    for (int i = 0; i < 3; i++) {
        x = i * 2;
        printf("x (inside loop) = %d\n", x);
    }
}

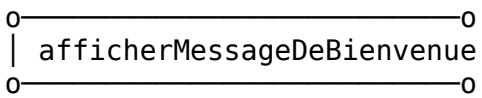
void main(void) {
    f();
}
```

La variable	est accessible de la ligne...	à la ligne...
i		
x		

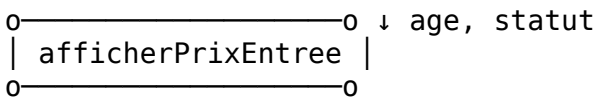
```
 1  void f() {
 2      int a = 10;
 3      printf("a (in function) = %d\n", a);
 4  }
 5
 6  void main(void) {
 7      int a = 5;
 8      printf("a (in main) = %d\n", a);
 9      f();
10  }
```

La variable	est accessible de la ligne...	à la ligne...
a (déclaration ligne 2)		
a (déclaration ligne 7)		

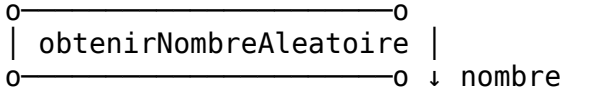
6.4. Pour chaque module, écris les prototypes des fonctions correspondantes. Enfin, indique quels sont les appels corrects et ceux qui ne le sont pas (écris « OK » ou « KO » à coté de chaque appel).



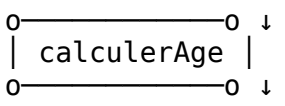

```
printf(afficherMessageDeBienvenue());
afficherMessageDeBienvenue;
afficherMessageDeBienvenue();
char msg = afficherMessageDeBienvenue();
printf("%s", afficherMessageDeBienvenue());
```




```
afficherPrixEntree();
afficherPrixEntree(17, 'E');
afficherPrixEntree('T', 17);
float prix = afficherPrixEntree(17, 'E');
```




```
obtenirNombreAleatoire(3);
obtenirNombreAleatoire();
int nombre = obtenirNombreAleatoire();
printf("%d", obtenirNombreAleatoire());
int nombre = obtenirNombreAleatoire() * 2;
```




```
printf("%d", calculerAge(1998,12,23));
calculerAge(1998,12,23);
int age = calculerAge(1998,12,23);
int age = calculerAge(19 * 10, 3 + 10, 100 – 2);
```

6.5. Complète le tableau suivant

Prototype	Appel
`void afficherMessage(void);`	
	`int somme = addition(5, 7);`
`void afficherNombre(int n);`	
	`int carre = calculerCarre(4);`
`bool estPositif(int nombre);`	
	`bool estValide = verifValid('A');`
	`float resultat = puissance(2.5, 3);`

N'oublie pas de tester toutes tes fonctions en les appelant dans la fonction principale main

6.6. Écris une fonction qui retourne la valeur de la somme de cinq nombres fournis en argument. Testez cette fonction.

6.7. Ecris une fonction « exposant » qui calcule l'exposant d'un nombre donné.
La fonction possède deux arguments. Le premier représente la base, le second l'exposant.
La fonction retourne en résultat le résultat de base ^{exposant}
Dans le programme principal :

- Demandez à l'utilisateur d'insérer un nombre pour la base, un nombre pour l'exposant.
- Affichez le résultat de la fonction exposant.

6.8. Écris une fonction qui retourne le maximum entre deux entiers fournis en argument.

6.9. Écris une fonction qui retourne true si un entier est pair, false sinon.

6.10. Écris une fonction qui retourne la valeur absolue d'un nombre réel.

6.11. Écris une fonction qui prend un caractère en argument et retourne true s'il s'agit d'une voyelle (a, e, i, o, u, y), false sinon. La fonction doit fonctionner pour les majuscules et les minuscules.

6.12. Écris une fonction qui convertit une température de Celsius (reçue en argument) en Fahrenheit. La formule est : `F = C × 9/5 + 32`.

6.13. Écris une fonction qui affiche tous les diviseurs d'un entier positif fourni en argument.

6.14. Écris une fonction qui calcule le prix TTC à partir d'un prix HT et d'un taux de TVA (en pourcentage), tous deux fournis en argument.

6.15. La suite de Fibonacci est une suite d'entiers dans laquelle chaque terme est la somme des deux termes qui le précèdent. Elle commence par les termes 0 et 1 et ses premiers termes sont :

F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	...	F_n
0	1	1	2	3	5	8	13		$F_{n-1}+F_{n-2}$

Créez une fonction « fibonacci » qui calcule la valeur du $n^{ième}$ terme de la suite. Cette fonction sera codée de manière itérative.

Le programme principal, demandera une valeur à l'utilisateur, Affichera ensuite une fois le résultat de la fonction «fibonacci».

6.16. Dépassement. Écris une fonction qui retourne le nombre de chiffres d'un entier positif. Par exemple, 4527 contient 4 chiffres.

6.17. Dépassement. Écris une fonction qui retourne true si un entier positif est un nombre premier, false sinon.

6.18. Dépassement. Écris une fonction qui prend en argument trois coefficients a, b et c d'une équation du second degré ($ax^2 + bx + c = 0$) et retourne le nombre de solutions réelles (0, 1 ou 2). Tu peux créer une fonction auxiliaire pour calculer le discriminant.

7. Les fonctions et tableaux

7.1. Pour chaque programme, indique si les paramètres passés à la fonction sont passés par copie ou par référence. Indique également (sans t'aider de l'ordinateur) l'affichage que produira le programme.

```
void traiter(int valeur) {
    valeur = 42;
}

void main(void) {
    int nombre = 5;
    traiter(nombre);
    printf("%d\n", nombre);
}
```

```
int traiter(int valeur) {
    valeur = 42;
    return valeur;
}

void main(void) {
    int nombre = 5;
    nombre = traiter(nombre);
    printf("%d\n", nombre);
}
```

```
void traiter(float valeur) {
    valeur = 3.14;
}

void main(void) {
    float nombre = 1.23;
    traiter(nombre);
    printf("%.2f\n", nombre);
}
```

```
void traiter(int tableau[], int taille) {
    tableau[0] = 99;
}

void main(void) {
    int valeurs[3] = {1, 2, 3};
    traiter(valeurs, 3);
    printf("%d\n", valeurs[0]);
}
```

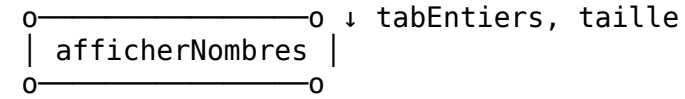
```c
void traiter(char caractere) {
    caractere = 'Z';
}

void main(void) {
    char lettre = 'A';
    traiter(lettre);
    printf("%c\n", lettre);
}
```

```c
void traiter(int valeur) {
    valeur = 99;
}

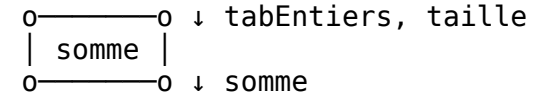
void main(void) {
    int valeurs[3] = {1, 2, 3};
    traiter(valeurs[0]);
    printf("%d\n", valeurs[0]);
}
```

7.2. Pour chaque module, écris les prototypes des fonctions correspondantes. Enfin, indique quels sont les appels corrects et ceux qui ne le sont pas (écris « OK » ou « KO » à coté de chaque appel).



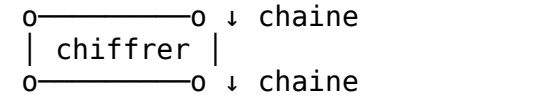

```
int tabEntiers[] = {10, 20, 30, 40, 50};


printf("%d", afficherNombres(tabEntiers, sizeof(tabEntiers));
int x = afficherNombres(tabEntiers, sizeof(tabEntiers));
afficherNombres(tabEntiers, sizeof(tabEntiers));
afficherNombres(tabEntiers, 5);
```




```
int tabEntiers[] = {10, 20, 30, 40, 50};


printf("%d", somme(tabEntiers, sizeof(tabEntiers));
int x = somme(tabEntiers, sizeof(tabEntiers));
somme(tabEntiers, sizeof(tabEntiers));
somme(tabEntiers, 5);
```




```
char chaine[] = "Hello !";


printf("%s", chiffrer(chaine, sizeof(chaine));
chiffrer(chaine, sizeof(chaine));
chiffrer(chaine[]);
chiffrer(chaine);
```

7.3. Complète le tableau suivant

Prototype	Appel
`void afficherTableau(int tab[], int taille);`	
	`float moyenne = calculerMoyenne(notes, 5);`
`bool rechercherElement(int tableau[], int taille, int valeur);`	
	`char lettre = obtenirCaractere(texte, 3);`
`int compterOccurrences(char texte[], char caractere);`	
`void trierTableau(float donnees[], int debut, int fin);`	
	`int maximum = trouverMax(nombres, 10);`
`float sommeTableau(float valeurs[], int taille);`	

7.4. Quel sera la sortie des programmes suivants ?

(a) Sortie simple

```c
void modifier(int x) {
    x = 10;
}

void main(void) {
    int a = 5;
    modifier(a);
    printf("%d\n", a);
}
```

(b) Modification d'un tableau

```c
void modifier(int tab[]) {
    tab[0] = 100;
}

void main(void) {
    int nombres[3] = {1, 2, 3};
    modifier(nombres);
    printf("%d\n", nombres[0]);
}
```

(c) Combinaison simple

```c
void traiter(int x, int tab[]) {
    x = 50;
    tab[1] = 50;
}

void main(void) {
    int val = 10;
    int data[3] = {1, 2, 3};
    traiter(val, data);
    printf("%d %d\n", val, data[1]);
}
```

(d) Boucle et tableaux

```c
void doubler(int tab[], int taille) {
    for (int i = 0; i < taille; i++) {
        tab[i] *= 2;
    }
}

void main(void) {
    int nums[4] = {1, 2, 3, 4};
    doubler(nums, 4);
    printf("%d %d\n", nums[0], nums[3]);
}
```

(e) Paramètre de taille modifié

```c
void modifier(int tab[], int n) {
    n = 10;
    tab[0] = n;
}

void main(void) {
    int tableau[3] = {5, 5, 5};
    int nombre = 3;
    modifier(tableau, nombre);
    printf("%d %d\n", tableau[0], nombre);
}
```

(f) Des appels à la pelle

```c
void incrementer(int x) {
    x++;
}

void ajouterUn(int tab[], int taille) {
    for (int i = 0; i < taille; i++) {
        tab[i]++;
    }
}

void main(void) {
    int val = 0;
    int nums[2] = {0, 0};

    incrementer(val);
    ajouterUn(nums, 2);
    incrementer(val);
    ajouterUn(nums, 2);

    printf("%d %d %d\n", val, nums[0], nums[1]);
}
```

(g) Echange raté

```
void echanger(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

void inverserTableau(int tab[]) {
    int temp = tab[0];
    tab[0] = tab[1];
    tab[1] = temp;
}

void main(void) {
    int x = 5, y = 10;
    int nums[2] = {5, 10};

    echanger(x, y);
    inverserTableau(nums);

    printf("%d %d %d %d\n", x, y, nums[0], nums[1]);
}
```

(h) Modification partielle

```
void traiter(int tab[], int debut, int fin) {
    debut = 0;
    fin = 10;
    for (int i = debut; i < fin && i < 5; i++) {
        tab[i] += i;
    }
}

void main(void) {
    int data[5] = {10, 10, 10, 10, 10};
    int a = 1, b = 3;
    traiter(data, a, b);
    printf("%d %d %d %d %d\n", data[0], data[1], data[2], data[3],
data[4]);
}
```

(i) Chaine de modifications

```c
void f1(int x, int tab[]) {
    x = 100;
    tab[0] = x;
}

void f2(int tab[]) {
    int local = tab[0];
    local = 200;
    tab[1] = local;
}

void f3(int y) {
    y = 300;
}

void main(void) {
    int val = 0;
    int nums[3] = {0, 0, 0};

    f1(val, nums);
    f2(nums);
    f3(nums[2]);

    printf("%d %d %d %d\n", val, nums[0], nums[1], nums[2]);
}
```

(j) Le piège ultime

```c
void mystere(int tab1[], int tab2[], int n) {
    n = n * 2;
    for (int i = 0; i < n/2; i++) {
        int temp = tab1[i];
        tab1[i] = tab2[i];
        tab2[i] = temp + i;
    }
}

void main(void) {
    int a[3] = {1, 2, 3};
    int b[3] = {10, 20, 30};
    int taille = 3;

    mystere(a, b, taille);

    printf("%d %d %d\n", a[0], a[1], a[2]);
    printf("%d %d %d\n", b[0], b[1], b[2]);
    printf("%d\n", taille);
}
```

7.5. Écris une fonction qui renvoie le nombre de voyelles contenues dans une chaîne de caractères passée en argument. Testez cette fonction.

7.6. Écris une fonction booléenne qui retourne un booléen à true si un tableau d'entiers est trié, à false

si le tableau n'est pas trié. Testez la fonction. La fonction possède comme argument le tableau, la longueur du tableau.

7.7. Écrivez une fonction qui nous indique si un tableau d'entiers est trié. En argument, cette méthode possède le tableau, la longueur du tableau, un booléen qui nous indique si on doit tester le tri en ordre croissant (true) ou en ordre décroissant (false).

7.8. Ecris une fonction « palindrome », cette fonction contient un paramètre : le mot à analyser. (le mot ne contient pas d'espace.) La fonction retourne true si le mot est un palindrome, false si la fonction n'est pas un palindrome. Dans le programme principal : demandez un mot à l'utilisateur. Testez ensuite la fonction palindrome. Si elle retourne une valeur vraie, indiquez à l'écran: « le mot est un palindrome ». Si elle retourne une valeur fausse indiquez à l'écran : « le mot n'est pas un palindrome ».

7.9. Crée un programme qui permet de lancer les deux exercices. Pour cela, créez une nouvelle fonction « interfacePrincipale » qui appelle le palindrome ou la suite de fibonacci. en fonction du choix de l'utilisateur. Si l'utilisateur entre autre chose que 1 ou 2, le programme répondra. « choix incorrect, veuillez réessayer ». À la fin de chaque exercice, introduis un extrait de code qui permette de revenir au menu principal. Voici un exemple d'interface :

```
--------------------------------
Bienvenue sur le menu principal
1. exercice palindrome ")
2. exercice fibonacci ")
--------------------------------

Choisissez le numéro de l'exercice :
```

7.10. Écris une fonction remplir qui prend en argument un tableau d'entiers et sa longueur, et qui remplace chaque élément par sa valeur absolue. Teste la fonction en affichant le tableau avant et après l'appel.

7.11. Écris une fonction `saisir_nom` qui prend en argument un tableau de caractères (vide) et qui demande à l'utilisateur de saisir son nom pour le stocker dans ce tableau. Écris une deuxième fonction `saluer` qui prend ce même tableau en argument et affiche « Bonjour <nom> ! ». Teste ces deux fonctions dans main.

7.12. Écris une fonction qui prend en argument un tableau d'entiers, sa longueur, et un entier `valeur`. La fonction remplace toutes les occurrences de `valeur` dans le tableau par 0.

7.13. Écris une fonction qui prend en argument un tableau d'entiers et sa longueur, et qui multiplie chaque élément du tableau par 2. Teste la fonction en affichant le tableau avant et après l'appel.

7.14. Écris une fonction qui prend en argument un tableau d'entiers et sa longueur, et qui inverse l'ordre des éléments du tableau (le premier devient le dernier, etc.) sans utiliser de tableau auxiliaire.

7.15. Écris une fonction qui prend en argument un tableau d'entiers, sa longueur, et un entier `valeurRemplacement`. La fonction remplace tous les entiers négatifs du tableau par `valeurRemplacement`.

7.16. Écris une fonction qui prend en argument un tableau d'entiers et sa longueur, et qui décale tous les éléments d'une position vers la gauche (le premier élément est perdu, et un 0 est inséré en dernière position).

7.17. Écris une fonction qui prend en argument un tableau d'entiers et un pointeur vers sa longueur. La fonction supprime les doublons du tableau en conservant la première occurrence de chaque valeur, et met à jour la longueur en conséquence.

7.18. Dépassement. Écris une fonction qui prend en argument un tableau d'entiers, sa longueur, et un booléen croissant. La fonction trie le tableau par ordre croissant si croissant vaut true, ou par ordre décroissant sinon, en utilisant l'algorithme du tri à bulles.

7.19. Dépassement. Écris une fonction fusionner qui prend en argument deux tableaux d'entiers triés en ordre croissant (avec leurs longueurs respectives) et un troisième tableau destiné à recevoir le résultat. La fonction fusionne les deux tableaux triés en un seul tableau trié.

7.20. Dépassement. Écris une fonction rotation qui prend en argument un tableau d'entiers, sa longueur, un entier k et un booléen gauche. La fonction effectue une rotation de k positions vers la gauche si gauche vaut true, ou vers la droite sinon. Les éléments qui "sortent" d'un côté réapparaissent de l'autre. Teste la fonction avec différentes valeurs de k.

```c
#include <stdio.h>
#include <stdlib.h>  // abs()
#include <string.h>  // strlen()
#include <stdbool.h> // bool

// ============================================================
// 7.10 - Remplacer chaque élément par sa valeur absolue
// ============================================================
void valeur_absolue(int tab[], int longueur) {
    for (int i = 0; i < longueur; i++) {
        tab[i] = abs(tab[i]);
    }
}

// ============================================================
// 7.11 - Saisir un nom et saluer
// ============================================================
void saisir_nom(char nom[]) {
    printf("Entrez votre nom : ");
    scanf("%s", nom);
}

void saluer(char nom[]) {
    printf("Bonjour %s !\n", nom);
}

// ============================================================
// 7.12 - Remplacer toutes les occurrences d'une valeur par 0
// ============================================================
void remplacer_valeur(int tab[], int longueur, int valeur) {
    for (int i = 0; i < longueur; i++) {
        if (tab[i] == valeur) {
            tab[i] = 0;
        }
    }
}

// ============================================================
// 7.13 - Multiplier chaque élément par 2
// ============================================================
void doubler(int tab[], int longueur) {
    for (int i = 0; i < longueur; i++) {
        tab[i] *= 2;
    }
}

// ============================================================
// 7.14 - Inverser l'ordre des éléments
// ============================================================
void inverser(int tab[], int longueur) {
    int temp;
    for (int i = 0; i < longueur / 2; i++) {
        temp = tab[i];
        tab[i] = tab[longueur - 1 - i];
        tab[longueur - 1 - i] = temp;
    }
```

```c
}

// ============================================================
// 7.15 - Remplacer les entiers négatifs par valeurRemplacement
// ============================================================
void remplacer_negatifs(int tab[], int longueur, int valeurRemplacement) {
    for (int i = 0; i < longueur; i++) {
        if (tab[i] < 0) {
            tab[i] = valeurRemplacement;
        }
    }
}

// ============================================================
// 7.16 - Décaler tous les éléments d'une position vers la gauche
// ============================================================
void decaler_gauche(int tab[], int longueur) {
    for (int i = 0; i < longueur - 1; i++) {
        tab[i] = tab[i + 1];
    }
    tab[longueur - 1] = 0;
}

// ============================================================
// 7.17 - Supprimer les doublons
// ============================================================
void supprimer_doublons(int tab[], int *longueur) {
    for (int i = 0; i < *longueur; i++) {
        for (int j = i + 1; j < *longueur; j++) {
            if (tab[j] == tab[i]) {
                // Décaler tous les éléments après j vers la gauche
                for (int k = j; k < *longueur - 1; k++) {
                    tab[k] = tab[k + 1];
                }
                (*longueur)--;
                j--; // Revenir en arrière car les éléments ont été décalés
            }
        }
    }
}

// ============================================================
// 7.18 - Tri à bulles croissant ou décroissant
// ============================================================
void trier_bulles(int tab[], int longueur, bool croissant) {
    int temp;
    for (int i = 0; i < longueur - 1; i++) {
        for (int j = 0; j < longueur - 1 - i; j++) {
            bool doitEchanger;
            if (croissant) {
                doitEchanger = tab[j] > tab[j + 1];
            } else {
                doitEchanger = tab[j] < tab[j + 1];
            }
            if (doitEchanger) {
                temp = tab[j];
```

```
                tab[j] = tab[j + 1];
                tab[j + 1] = temp;
            }
        }
    }
}

// ============================================================
// 7.19 - Fusionner deux tableaux triés en un seul tableau trié
// ============================================================
void fusionner(int tab1[], int longueur1, int tab2[], int longueur2, int
resultat[]) {
    int i = 0, j = 0, k = 0;
    while (i < longueur1 && j < longueur2) {
        if (tab1[i] <= tab2[j]) {
            resultat[k++] = tab1[i++];
        } else {
            resultat[k++] = tab2[j++];
        }
    }
    // Copier les éléments restants de tab1 s'il en reste
    while (i < longueur1) {
        resultat[k++] = tab1[i++];
    }
    // Copier les éléments restants de tab2 s'il en reste
    while (j < longueur2) {
        resultat[k++] = tab2[j++];
    }
}

// ============================================================
// 7.20 - Rotation de k positions vers la gauche ou la droite
// ============================================================
void rotation(int tab[], int longueur, int k, bool gauche) {
    k = k % longueur; // Éviter les rotations inutiles
    if (!gauche) {
        k = longueur - k; // Une rotation droite de k = rotation gauche de
(n-k)
    }
    // Rotation gauche de k positions avec 3 inversions
    // 1. Inverser les k premiers éléments
    for (int i = 0; i < k / 2; i++) {
        int temp = tab[i];
        tab[i] = tab[k - 1 - i];
        tab[k - 1 - i] = temp;
    }
    // 2. Inverser les éléments restants
    for (int i = k; i < k + (longueur - k) / 2; i++) {
        int temp = tab[i];
        tab[i] = tab[longueur - 1 - (i - k)];
        tab[longueur - 1 - (i - k)] = temp;
    }
    // 3. Inverser le tableau entier
    for (int i = 0; i < longueur / 2; i++) {
        int temp = tab[i];
        tab[i] = tab[longueur - 1 - i];
```

```c
        tab[longueur - 1 - i] = temp;
    }
}

// ============================================================
// Fonction utilitaire : afficher un tableau
// ============================================================
void afficher_tableau(int tab[], int longueur) {
    printf("[");
    for (int i = 0; i < longueur; i++) {
        printf("%d", tab[i]);
        if (i < longueur - 1) printf(", ");
    }
    printf("]\n");
}
```

8. Les tableaux 2D

9. Synthèse des compétences

Créez un programme qui contienne 2 tableaux.

- Le premier contiendra le nombre de tours effectués.
- Le deuxième contiendra l'écurie.

Afin de garder un cohésion entre les deux tableaux. **Les index de ceux-ci feront à chaque fois référence au numéro de la voiture**.

1 L'objectif de l'exercice sera le suivant :

- Le programme demande à l'utilisateur d'insérer un numéro de voiture.
- Si le numéro de la voiture n'est pas correct, le programme indiquera un message d'erreur.
- Si le numéro est correct, le logiciel affiche, le nombre de tours effectués ainsi que l'écurie de la voiture.
- Avant de s'interrompre, le logiciel demande à l'utilisateur s'il veut demander les informations d'une autre voiture. Réponse par O / N.

Voiture	Equipe / Ecurie	Nombre de tours
0	Undefined	-1
1	Porsche Team	346
2	Signatech Alpine	357
3	Toyota Gazoo Racing	381
4	Audi Sport Team Joest	372
5	Race Performance	297
6	G-Drive Racing	353
7	Murphy Prototypes	323
8	SO24! by Lombard Racing	328
9	Greaves Motorsport	348
10	Strakka Racing	351
11	AF Corse	179
12	Manor	283

2 L'objectif de l'exercice sera le suivant :

Le programme affichera les informations de l'ensemble des voitures sous un dialogue tel que :

```
voiture : 1 | equipe : Porsche Team | tours : 346
voiture : 2 | equipe : Signatech Alpine | tours : 357
voiture : 3 | equipe : Toyota Gazoo Racing | tours : 381
.....
```

3 L'objectif de l'exercice sera le suivant : après une recherche dans les tableaux, le logiciel affichera le podium de la course.

4 Crée une interface qui donne accès à chacun des exercices. Exemple de dialogue :

```
_---------------------------------
1. Rechercher une voiture
2. Afficher la liste des voitures
3. Afficher le podium
_---------------------------------
Choisissez un numero 1-3 : 5
Numero incorrect !
Choisissez un numero 1-3 : _
```

Adaptez les exercices afin de permettre le retour au menu principal. Exemple de dialogue

```
Pour retourner au menu principal, tapez Q
```

10. Les pointeurs

10.1. Observe le code suivant :

```
char u, v = 'E';
char *pu, *pv = &v;

*pv = v – 3;
u = *pv + 2;
pu = &u + 1;
```

Sachant que la variable `u` se trouve à l'adresse `A1B` et v à l'adresse `A1C`, que valent :

- `*pv ?`
- `u ?`
- `pu ?`
- `*pu ?`

10.2. Observe le code suivant :

```
float a = 12.4;
float b = -3.6;
float c;
float *pa = &a
float *pb = &b;
```

Sachant que a, b et c se trouvent respectivement aux adresses 2471, 2138 et 1998, quel sera l'effet des instructions suivantes :

- `*pa = 5*a`
- `c = 2*(*pa - *pb)`
- `--*pb`
- `(*pa)++`
- `pa++`

10.3. Écris une fonction nommée « initialisation » qui se charge d'initialiser trois variables entières (`n1`, `n2`, `n3`) à 0.

10.4. Observe le code suivant :

```
int tabNbr[] = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
int *pTabNbr =  tabNbr;
int i = 2;
```

Pour chaque expression, indique son type et sa valeur ainsi que la valeur de `pTabNbr` après l'exécution.

Expression	Type de l'expression	Valeur de l'expression	Valeur de `pTabNbr` après exécution
`pTabNbr`			
`*pTabNbr`			
`++pTabNbr`			
`(*pTabNbr)--`			
`*(++pTabNbr)`			
`pTabNbr + i`			
`(*pTabNbr + i)`			
`*(i + pTabNbr)`			

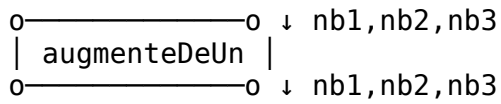
10.5. Écris une fonction qui calcule la moyenne des éléments d'un tableau d'entiers sachant que son prototype est :

```
float moyenne (int *pTab, int taille);
```

Ensuite, écris la fonction `main` dans laquelle tu déclares un tableau de 5 notes (entières) ainsi qu'une variable `moyenneNotes`. Fait en sorte que cette variable contienne le résultat de la fonction `moyenne`.

10.6. Écris une fonction qui reçoit une chaîne de caractères et qui renvoie `true` si cette chaîne contient au moins un chiffre et `false` sinon. Attention, tu ne peux ni utiliser la fonction `strlen` ni les constantes symboliques, ni les crochets (`[]`).

10.7. Quels sont l'appel et le prototype de la fonction suivante ?



10.8. Observe le code suivant :

```
char chaine[] = "l'INDBG c'est cool !";
char *p = chaine + 8;
```

Quel sera l'affichage produit par chacune des instructions **successives** suivantes ?

- `printf("%c\n",*p);`
- `printf("%s\n", ++p);`
- `printf("%c\n",++(*p));`
- `printf("%s\n", p);`
- `printf("%c\n",*--p);`
- `printf("%c\n",*p+2);`
- `printf("%c\n",*p);`

10.9. Ecris une fonction qui ajoute un tableau de réels à la suite des réels déjà présents dans un autre tableau. Le premier tableau est suffisamment long pour y ajouter le second tableau. Le prototype est :

```
void concatenation(float *pTab1, float *pTab2, int tailleTab1, int
tailleTab2);
```

10.10. Ecris une fonction qui permet de connaître le nombre d'occurrences de chacune des lettres de l'alphabet dans un texte.

11. Les structures

11.1. On te demande de concevoir un programme permettant l'encodage de plusieurs élèves. Pour chaque élève, on souhaite retenir son nom, son prénom, son age, son sexe ainsi que sa moyenne globale obtenue durant chaque année scolaire.

a. Ecris la structure correspondante.

b. Déclare un type (synonyme) pour cette structure.

c. Déclare et initialise une variable nommée `jean` de ce type contenant les informations de Jean Dubois (de ton choix).

d. Réécris la structure (ajoute des nouvelles structures et synonymes si c'est nécessaire) : pour chaque année scolaire, on souhaite retenir la moyenne obtenue ainsi que l'identifiant de sa classe (ex. « 4G1 », « 5G2 », etc.)

e. Ecris une fonction qui se charge d'afficher les informations d'un élève (reçu en argument) de la manière suivante (les parties soulignées doivent varier en fonction des infos de l'élève) :

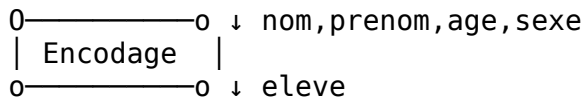
```
Fiche de l'élève : Jean Dubois (H)
- 1C : 70/100
- 2C : 65/100
- 3G1 : 78/100
- 4G2 : 73/100
```

f. Déclare un tableau contenant 5 élèves de ton choix et pouvant contenir 10 élèves au maximum.

g. Ecris une fonction qui se charge d'ajouter un nouvel élève (reçu en argument) dans le tableau d'élèves (reçu en argument).

h. Ecris l'instruction permettant d'enregistrer une nouvelle moyenne à Jean : en 5G1, il a obtenu 68/100.

11.2. On souhaite créer un programme permettant l'encodage d'un élève. On te demande d'écrire la fonction correspondante au module suivant :



La fonction encodage prend en entrée une série d'information sur l'élève et retourne une structure

11.3. On souhaite gérer des produits dans un entrepôt. Un produit est défini par 3 critères:

- l'entier code produit ("567" par exemple qui est un nombre entre 255 et 450),
- l'intitulé ("pots de peinture" et qui comporte au maximum 99 caractères utiles)
- un entier qui indique la quantité en stock (803 par exemple).

Le tableau des produits comportera au maximum 100 produits.

Il faut gérer le tableau grâce au menu suivant :

1. Ajouter un produit (on tape le code produit et l'intitulé, la quantité et le stock).

2. Afficher la liste de produits.
3. Modifier un élément existant.
On veillera à bien décomposer ce problème en différents modules et à mener une réflexion sur les fonctions nécessaires dans chaque module.

Par défaut, on estime que les insertions faites par l'utilisateur sont correctes.

Dépassement : Il faut gérer une liste de produits en veillant à ce qu'il n'y ait pas deux produits avec le même code produit. Bien sûr la quantité en stock ne peut pas être négative.

L'interface principale renvoie vers trois procédures distinctes :

- menuAjouter
- menuResume
- menuModifier

Celles-ci doivent parfois également utiliser des sous-fonctions. Notamment pour la recherche d'indice dans menuModifier.

12. Les fichiers

12.1. Tu est responsable de la base de données des produits d'une grande surface. Un article est caractérisé par un libellé (ex. « Lait entier ») et un prix (ex. 1,50€). Les articles doivent être encodés dans un fichier binaire (enregistrements de longueur fixe).

a) Déclare la structure correspondante

b) Déclare un pointeur permettant d'avoir **accès en écriture** aux informations enregistrées dans le fichier

c) Ajoute plusieurs articles :

i. « Croissant » au prix de 0,50€

ii. « Lait entier » au prix de 1,20€

iii. « Compote » au prix de 2,20€

iv. « Pain complet » au prix de 2,50€

d) Ferme le fichier puis rouvre-le afin d'y avoir **accès uniquement en lecture**

e) Affiche la liste des articles

f) Ferme le fichier puis rouvre-le afin d'y avoir **accès en lecture et écriture** (sans écraser le fichier)

g) Modifie le prix de la compote pour qu'il soit de 2,50€

h) Ferme le fichier puis rouvre-le afin d'y avoir **accès uniquement en lecture**

i) Affiche la liste des articles et vérifie que le prix de la compote a bien été modifié

12.2. Écris un programme qui crée un fichier `avis_restaurants.dat` qui reprend des données sur des restaurants. Par restaurant, nous souhaitons connaître le nom du restaurant (max. 100 caractères), le nombre d'avis donnés (un entier) ainsi que la moyenne des notes attribuées de 0 à 5 (un réel). Chaque enregistrement est de longueur fixe. Ton programme doit fournir 2 fonctionnalités (sous forme d'un menu) :

a) la création du fichier avec l'encodage de plusieurs restaurants (le nombre d'avis et la moyenne des notes doivent être initialisées à 0 automatiquement)

b) l'affichage des restaurants sous la forme
<nom> (<moyenne>/5 - <nbAvis> avis)

12.3. Ajoute une nouvelle fonctionnalité au programme réalisé dans l'exercice précédent : l'utilisateur doit pouvoir ajouter une note à un restaurant existant. La moyenne doit alors être automatiquement recalculée et mise à jour. Le nom du restaurant peut ne pas être dans le fichier : dans ce cas, il faut afficher un message d'erreur. Par facilité, les entrées de l'utilisateur (nom du restaurant et note) doivent être stockées dans une structure.

12.4. Ajoute trois nouvelles fonctionnalités :

a) Ajouter un restaurant à la fin du fichier (sans écraser le fichier existant)

b) Supprimer un restaurant (par facilité, le nom est simplement remplacé par « *** » ; l'enregistrement est encore physiquement présent dans le fichier). Attention, fais en sorte que les restaurants supprimés ne s'affichent pas dans la liste des restaurants

c) Modifier le nom d'un restaurant